

Buggy-Path-First: A Risk-Guided Search Strategy for Symbolic Execution

Kunlong He
Simon Fraser University
kha112@sfu.ca

Abstract

Symbolic execution is a powerful technique for program analysis, but it often suffers from the path explosion problem [1], which makes bug discovery inefficient. In this project, I propose a new search strategy for KLEE called Buggy-Path-First. The main idea is to assign a risk score to each execution state based on runtime features such as symbolic memory access, pointer usage, and branch complexity, and then prioritize states with higher scores. I implement this strategy as a lightweight extension of KLEE’s existing search framework. The evaluation shows that Buggy-Path-First performs similarly to existing strategies on simple benchmarks, and achieves better performance on more complex programs with misleading paths and memory-related behavior, reducing the time to first bug. These results suggest that guiding exploration with simple risk signals can improve efficiency in certain types of programs, especially those with misleading paths and memory-related behavior.

1 Introduction

In this project, I focus on improving the efficiency of bug finding in symbolic execution. Symbolic execution tools like KLEE [2] are powerful, but they often suffer from the path explosion problem. As the number of program paths grows very fast, the tool may spend a lot of time exploring paths that are unlikely to contain bugs [3]. Because of this, important bugs may be discovered very late, or even missed within limited time and resources. I believe this is an important problem, since faster bug discovery can save both time and computing cost, and make symbolic execution more practical for real-world programs.

Currently, KLEE provides several search strategies, such as depth-first search (DFS), random search, and non-uniform random search (NURS)-based strategies [2]. These methods mainly focus on exploring paths in a general or coverage-oriented way. For example, DFS goes deep into one path, random search selects states randomly, and NURS tries to im-

prove coverage by assigning weights. However, none of these strategies directly consider how likely a path is to contain a bug. As a result, they may spend too much time exploring safe or unimportant paths, which does not match my goal of finding bugs as early as possible.

To address this limitation, I implement a new search strategy in KLEE called Buggy-Path-First, inspired by AEG: Automatic Exploit Generation [4]. The main idea is to prioritize execution states that are more likely to lead to bugs. I design a simple risk scoring mechanism based on features such as symbolic memory operations, pointer usage, and complex branch conditions. The searcher always selects the state with the highest risk score first. In this way, the exploration is guided toward potentially buggy paths earlier [5].

My goal is to reduce the time to find bugs and improve the overall efficiency of symbolic execution. Compared with existing strategies, my approach directly targets bug discovery instead of general exploration or coverage. This helps fill the gap where current methods do not explicitly prioritize risky paths. By doing this, I expect the system to find bugs earlier, making symbolic execution more efficient in practice.

2 Design Choices

I designed Buggy-Path-First as a lightweight extension of KLEE’s existing search framework, so that it can be integrated without changing the overall execution model. Instead of rewriting the executor, I added a new search strategy, BuggyPathFirstSearcher, by extending KLEE’s core searcher interface and implementation, and registered it through the option `-search=buggy-path-first` [6]. This design makes the new strategy easy to compare with existing KLEE searchers such as DFS, random-path, and NURS-based strategies.

The main design choice was to rank states by a bug-oriented heuristic instead of a coverage-oriented one. For this reason, I added a `riskScore` field to `ExecutionState` in `lib/Core/ExecutionState.h`, together with several counters such as `symbolicMemoryAccesses` and `symbolicPointerUses`. I then implemented `computeRiskScore()` and re-

freshRiskScore() in lib/Core/ExecutionState.cpp. The scoring function gives higher priority to states that interact more often with symbolic memory and symbolic pointers, encounter more complex branch conditions, or get closer to memory-related failures. I also kept a small contribution from depth, so that deeper suspicious paths are not ignored completely.

I define the risk score as the sum of several bug-related signals, each with a fixed weight:

Metric	Example	Score
Sym. Memory Access	buf[i]	20
Sym. Pointer Use	*p	30
Complex Branch	if (x > y)	2
Memory Error	OOB / null deref	100
Depth	deeper path	1

I set the weight of symbolic memory access to 20 and symbolic pointer use to 30 because I wanted symbolic pointer behavior to have a slightly stronger influence than general symbolic memory access. I set the weight of complex branch score to 2 because branch complexity is only an indirect signal, so I did not want it to dominate the ranking. I set the weight of memory-related error events to 100 because they are the strongest sign of bug-prone execution in this heuristic. I also kept depth with weight 1 as a small extra factor, so deeper suspicious paths are not ignored. These values were chosen as practical heuristic settings to reflect relative importance [7], not as mathematically optimal constants.

Another design choice was to collect risk signals at places where bugs are more likely to occur. Instead of defining these signals in advance, I integrated their collection into the executor so the risk score can be updated during execution. When KLEE observes relevant runtime events, it updates the state accordingly.

Finally, I implemented the scheduler itself with a priority queue [4]. BuggyPathFirstSearcher stores states as QueueEntry objects and always selects the state with the highest riskScore in selectState(). I also used pushOrRefresh() and discardStaleEntries() to keep the queue consistent when states are updated or removed. I chose this design because it is simple, efficient, and matches the goal of always exploring the currently most suspicious path first.

3 Algorithm

I implemented Buggy-Path-First as a risk-guided search strategy for KLEE. The core idea is simple: instead of exploring states in a generic order, it gives each ExecutionState a risk score and prefers states with higher scores. The method has two main parts: risk score updating and state scheduling.

Risk Score Updating. Each state stores several counters that describe behaviors related to bug risk, and these counters are updated at specific points during execution. For example, the counters increase when KLEE sees symbolic memory

Algorithm 1 UpdateRiskScore

```

1: procedure COMPUTE_RISK_SCORE(state)
2:   return  $20 \cdot \text{state.symbolicMemoryAccesses}$ 
3:      $+ 30 \cdot \text{state.symbolicPointerUses}$ 
4:      $+ 2 \cdot \text{state.complexBranchScore}$ 
5:      $+ 100 \cdot \text{state.memoryErrorEvents}$ 
6:      $+ \text{state.depth}$ 
7: end procedure

8: procedure UPDATE_ONE_EVENT(state, event, value)
9:   if event = symbolicMemoryAccess then
10:    state.symbolicMemoryAccesses  $\leftarrow$  state.symbolicMemoryAccesses + 1
11:   else if event = symbolicPointerUse then
12:    state.symbolicPointerUses  $\leftarrow$  state.symbolicPointerUses + 1
13:   else if event = complexBranch then
14:    state.complexBranchScore  $\leftarrow$  state.complexBranchScore + value
15:   else if event = memoryError then
16:    state.memoryErrorEvents  $\leftarrow$  state.memoryErrorEvents + 1
17:   end if
18:   state.riskScore  $\leftarrow$  COMPUTE_RISK_SCORE(state)
19: end procedure

```

accesses, symbolic pointer uses, complex symbolic branch conditions, or certain memory-related program errors. After one of these counters changes, the state recomputes its riskScore immediately.

State Scheduling. Active states are managed by BuggyPathFirstSearcher using a priority queue ordered by riskScore. States with higher scores are given higher priority. When the scheduler needs a state, it chooses the valid state with the highest current score.

When a new state is created, it is inserted into the queue. During the normal searcher update process, the current active state is also refreshed and inserted again with its latest score. Because of this, the same state may appear in the queue more than once.

To handle repeated entries, the scheduler uses a generation number for each state. Every time a state is inserted or refreshed, its generation number is increased. Each queue entry stores both the state pointer and the generation number. Before returning the top state, the scheduler checks whether the queue entry is still current. If the entry is stale, it is removed. This continues until a valid entry is found.

Update Process. The scheduler is updated as part of KLEE’s normal execution flow. States that are removed from execution are deleted from the active set and will not be scheduled again. Newly added states are inserted into the queue. If the current state is still active, it is refreshed so that its latest score can affect future scheduling.

Selection of Next State. To select the next state, the scheduler first removes stale queue entries from the top. It then returns the valid state with the highest riskScore. In this way, Buggy-Path-First focuses exploration on states that currently look more suspicious.

Algorithm 2 Buggy-Path-First Searcher

```

1: Data:
2:   priority queue  $Q$  ordered by larger riskScore first
3:   set ActiveStates
4:   map Generations[state]
5: procedure PUSHORREFRESH(state)
6:   insert state into ActiveStates
7:   Generations[state]  $\leftarrow$  Generations[state] + 1
8:   entry  $\leftarrow$  (state.riskScore, Generations[state], state)
9:   push entry into  $Q$ 
10: end procedure
11: procedure DISCARDSTALEENTRIES
12:   while  $Q$  is not empty do
13:     top  $\leftarrow$   $Q$ .top()
14:     if top.state is in ActiveStates and Generations[top.state] = top.generation then
15:       break
16:     end if
17:     pop  $Q$ 
18:   end while
19: end procedure
20: procedure SELECTSTATE
21:   DISCARDSTALEENTRIES
22:   return  $Q$ .top().state
23: end procedure
24: procedure UPDATE(current, addedStates, removedStates)
25:   for all state in removedStates do
26:     remove state from ActiveStates
27:     erase Generations[state]
28:   end for
29:   if current exists and current not in removedStates then
30:     PUSHORREFRESH(current)
31:   end if
32:   for all state in addedStates do
33:     PUSHORREFRESH(state)
34:   end for
35: end procedure

```

4 Benchmarks

I selected seven benchmarks to evaluate Buggy-Path-First. Three of them are core benchmarks from KLEE: DivCheck, Malloc, and Searchers. Three others are benchmarks I de-

signed myself: DeepBug, MixedBug, and MixedBug2. I also included one adapted Juliet-style benchmark.

I chose these benchmarks because they cover different bug types and different path structures. DivCheck contains a simple divide-by-zero bug on a shallow path, so it is useful as a basic case. Malloc introduces dynamic memory allocation and a possible out-of-bounds write, which helps test memory-related bug discovery. Searchers places the bug behind several nested branches, making it a good example of a deeper bug.

My own benchmarks are designed to be more challenging. DeepBug uses a long chain of arithmetic conditions, so the bug is hidden behind a deep path with stronger constraints. MixedBug combines several patterns in one program, including deep arithmetic conditions, pointer and array access, a memory bug, a divide-by-zero bug, and also some safe branches that do not contain bugs. MixedBug2 is designed to mislead the search process: it starts with simple bug-free paths, while the real bug appears later in a memory-related path that also requires extra arithmetic conditions.

I also included a Juliet-style benchmark to provide a slightly more realistic memory-safety example. I adapted it from a public Juliet benchmark and simplified some environment-dependent parts, such as networking-related code, so that it could run more easily in KLEE. I kept the main bug pattern and the general control-flow structure. This benchmark is useful because it contains more realistic memory-related behavior than the smaller test examples, while still being manageable for comparison across search strategies.

5 Evaluation Results

For each benchmark, I first compile the C program into LLVM bitcode. For example, DivCheck.c is compiled with:

```
clang -I /home/klee/klee_src/include -emit-llvm
-c DivCheck.c -o DivCheck.bc
```

After that, KLEE runs the bitcode with different search strategies. For DFS, the command is:

```
klee --search=dfs --output-dir=dfs DivCheck.bc
```

For Buggy-Path-First, the command is:

```
klee --search=buggy-path-first --output-dir=buggy-path-first DivCheck.bc
```

The `--output-dir` option saves the results of each run in a separate folder, so the outputs of different strategies can be compared easily.

To reduce the effect of startup overhead and transient system noise, I performed a few warm-up runs before collecting the reported measurements. The warm-up runs were excluded from the final results.

I used the same benchmark input and the same basic execution setting for all strategies so that the comparison would be

fair. All experiments were run under the same experimental setup, using the same machine and intended software environment, including the same KLEE build, LLVM toolchain, and solver backend. I then compared the time to first bug and the number of completed paths across these runs.

The time function in this project is designed to measure time to first bug. When KLEE starts execution, the project saves the current wall-clock time as the start time in StatsTracker. Later, when the first bug is detected, KLEE calculates the elapsed time by subtracting the start time from the current time. I ran each strategy 10 times, then computed the mean and standard deviation, and recorded the results.

Search Strategy	Time to First Bug (s)	Completed Paths
DFS	0.103 ± 0.004	1
Random-Path	0.105 ± 0.006	1
NURS (covnew)	0.108 ± 0.005	1
Buggy-Path-First	0.097 ± 0.003	1

Table 1: Performance comparison on DivCheck

Search Strategy	Time to First Bug (s)	Completed Paths
DFS	0.253 ± 0.010	3
Random-Path	0.244 ± 0.012	3
NURS (covnew)	0.258 ± 0.011	3
Buggy-Path-First	0.231 ± 0.009	3

Table 2: Performance comparison on Malloc

Search Strategy	Time to First Bug (s)	Completed Paths
DFS	0.231 ± 0.008	4
Random-Path	0.224 ± 0.009	4
NURS (covnew)	0.222 ± 0.008	4
Buggy-Path-First	0.222 ± 0.007	4

Table 3: Performance comparison on Searchers

For DivCheck, Malloc, and Searchers in Tables 1–3, the running times are very close. I think this is expected. These three benchmarks are small, and their path spaces are limited. In DivCheck, the bug is on a very short and direct path, so almost any strategy can reach it quickly. In Malloc, although the program contains a memory-related bug, the whole benchmark is still simple, with only a few paths, so there is little room for the search strategy to make a big difference. In Searchers, the bug is deeper, but the structure is still very regular: the path is mostly a chain of nested conditions on one symbolic variable. Because of this, the built-in strategies can also reach the bug

without much difficulty, and Buggy-Path-First does not gain a strong advantage.

Search Strategy	Time to First Bug (s)	Completed Paths
DFS	25.43 ± 0.61	8
Random-Path	25.94 ± 0.74	8
NURS (covnew)	25.67 ± 0.68	8
Buggy-Path-First	25.56 ± 0.65	8

Table 4: Performance comparison on DeepBug

The result on DeepBug in Table 4 is also close to the built-in strategies. Although this benchmark is deeper and harder than the first three, it is still mainly a branch-constraint problem rather than a memory-behavior problem. The bug is hidden behind a long sequence of arithmetic conditions, but there are no symbolic pointers, no symbolic memory accesses, and no clear memory-error signals before reaching the bug. As a result, Buggy-Path-First cannot obtain much extra guidance from the features it uses most strongly. It can still benefit a little from path depth and branch complexity, but that advantage is limited, so the overall times remain similar.

Search Strategy	Time to First Bug (s)	Completed Paths
DFS	7.16 ± 0.42	105
Random-Path	1.29 ± 0.15	105
NURS (covnew)	2.27 ± 0.21	105
Buggy-Path-First	0.86 ± 0.09	105

Table 5: Performance comparison on MixedBug

Search Strategy	Time to First Bug (s)	Completed Paths
DFS	5.88 ± 0.37	60
Random-Path	5.61 ± 0.34	60
NURS (covnew)	6.52 ± 0.41	60
Buggy-Path-First	3.85 ± 0.26	60

Table 6: Performance comparison on MixedBug2

The difference becomes much clearer in MixedBug and MixedBug2, shown in Tables 5 and 6. I designed these two benchmarks to contain more misleading paths and more memory-related operations. This matches the design of Buggy-Path-First well. In MixedBug, the program mixes several kinds of behaviors: deep arithmetic conditions, pointer and array accesses, a divide-by-zero bug, safe branches, and a memory-related bug. This creates a larger and noisier search space. In this setting, Buggy-Path-First can focus earlier on states that look more risky, while DFS may spend time ex-

ploring less useful branches, and Random-Path or NURS may still be distracted by many non-buggy alternatives.

MixedBug2 shows this effect even more directly. The benchmark begins with several simple but harmless paths, while the real bug is placed later in a memory-related path with extra arithmetic checks. This means some strategies may spend time on the easy early paths even though they do not lead to a bug. Buggy-Path-First performs better because once execution reaches the pointer and memory operations, the corresponding states receive higher risk scores and are selected earlier. In other words, this benchmark is closer to the scenario that my strategy is designed for: the bug is not just deep, but also hidden behind memory-relevant behavior and misleading path structure.

Search Strategy	Time to First Bug (s)	Completed Paths
DFS	5.26 ± 0.410	1248
Random-Path	0.836 ± 0.072	1248
NURS (covnew)	0.761 ± 0.064	1248
Buggy-Path-First	0.578 ± 0.051	1248

Table 7: Performance comparison on Juliet

The Juliet benchmark in Table 7 was adapted from a public Juliet-style benchmark. I kept the core bug pattern and the general control-flow structure, but I removed some unrelated support code to make the test more suitable for KLEE-based evaluation. In particular, I removed external input and networking-related code such as socket, connect, and recv, because these parts introduce extra environment-dependent behavior that is not the main focus of this project. I then replaced them with simpler symbolic inputs so that the benchmark could focus more directly on how different search strategies explore risky paths. These changes make the benchmark easier to run and compare fairly, while still preserving the main memory-safety behavior that I want to evaluate.

The result in Table 7 shows that Buggy-Path-First is much faster than DFS on this benchmark. It is also slightly faster than Random-Path and NURS.

In addition, the number of completed paths is consistent across all strategies in the experiments. This suggests that Buggy-Path-First changes the exploration order without obviously affecting normal path exploration.

Besides the benchmarks above, I also attempted to evaluate the approach on larger real-world programs such as coreutils; however, due to limited hardware resources—particularly insufficient memory and computational capacity required for symbolic execution—I do not include those results in the evaluation.

6 Limitations

My current Buggy-Path-First strategy still has several limitations. First, its risk score is based on a small set of manually designed features, mainly symbolic memory access, symbolic pointer use, complex branches, memory-error signals, and path depth. Because of this, it works better on programs with memory-related risky behavior, but it may not show clear improvement on simple benchmarks or on bugs that are mostly hidden behind arithmetic constraints.

Second, the scoring method is fixed. I do not learn the weights automatically or adapt them to different program types. As a result, the strategy may perform well on some benchmarks but only similarly to existing KLEE strategies on others. In addition, the weights are manually chosen, so they are somewhat subjective and may not be fully accurate. They reflect my design judgment rather than an automatically learned or formally optimized result. Because of this, the scoring model may not always rank states in the best possible way.

Finally, my evaluation is still limited in scale. I tested the method on several small benchmarks, including my own designed cases, but I have not yet shown its effectiveness on larger real-world programs. In the future, I would like to test it on more complex benchmarks and improve the scoring model so that it can handle a wider range of bug patterns.

7 Conclusion

In this project, I implemented a new search strategy for KLEE called Buggy-Path-First. The main idea is to guide symbolic execution toward paths that are more likely to contain bugs, using a simple risk scoring mechanism. Compared with existing strategies, this approach focuses directly on bug discovery instead of general exploration.

The evaluation results show that Buggy-Path-First performs similarly to existing strategies on simple benchmarks, but achieves better performance on more complex programs with misleading paths and memory-related behavior. In particular, it reduces the time to first bug in the mixed-bug program, which suggests that the risk-guided heuristic can improve search efficiency in certain challenging cases.

References

- [1] C. Cadar and K. Sen, “Symbolic execution for software testing: three decades later,” *Communications of the ACM*, vol. 56, no. 2, pp. 82–90, 2013.
- [2] C. Cadar, D. Dunbar, D. R. Engler, *et al.*, “Klee: unassisted and automatic generation of high-coverage tests for complex systems programs,” in *OSDI*, vol. 8, pp. 209–224, 2008.

- [3] R. Baldoni, E. Coppa, D. C. D’elia, C. Demetrescu, and I. Finocchi, “A survey of symbolic execution techniques,” *ACM Computing Surveys (CSUR)*, vol. 51, no. 3, pp. 1–39, 2018.
- [4] T. Avgerinos, S. K. Cha, A. Rebert, E. J. Schwartz, M. Woo, and D. Brumley, “Automatic exploit generation,” *Communications of the ACM*, vol. 57, no. 2, pp. 74–84, 2014.
- [5] H. Tu, L. Jiang, and M. Böhme, “Vital: Vulnerability-oriented symbolic execution via type-unsafe pointer-guided monte carlo tree search,” *ACM Transactions on Software Engineering and Methodology*, 2024.
- [6] S. Andrica and G. Candea, “Pathscore-relevance: A metric for improving test quality,” in *Proceedings of the 5th Workshop on Hot Topics in System Dependability (Hot-Dep)*, 2009.
- [7] S. Saha, M. Downing, T. Brennan, and T. Bultan, “Preach: a heuristic for probabilistic reachability to identify hard to reach statements,” in *Proceedings of the 44th International Conference on Software Engineering*, pp. 1706–1717, 2022.
- [8] S. Yao and D. She, “Empc: Effective path prioritization for symbolic execution with path cover,” in *2025 IEEE Symposium on Security and Privacy (SP)*, pp. 2995–3013, IEEE, 2025.

[4] [1] [3] [2] [6] [7] [8] [5]